

Method Selection and Planning

Cohort 3

Group 11

Team Aubergine

Joshua Wainwright, Piotr Koziol, Sarvesh Sridhar, Harrison Barrans,
Daniel Thwaites, Callum Newton, Arnav Jamidar, Harry Turner

Justification of development & collaboration tools & methods:

Git

Our choice of Git as the version control system because of its distributed nature, which complements the team's Agile workflow. Git enables collaborative integration but also iterative updates, which allows the team to work independently around their timetables. Efficient branching and merging capabilities were key considerations as they allow for parallel development and experimentation, which can then be easily merged into the main project. This was essential as experimentation was a critical aspect of our development process. Further, integration with our teams' tooling, GitHub and IntelliJ IDE, helped streamline our process of propagating code. In contrast, the other main version control solution Subversion employs a centralised model that introduces a single point of failure, the server. Given the limited time frame we had, it would be too much of a risk. Git's distributed approach alleviated this via multiple redundant copies of the repository.

Github Issues

GitHub Issues was used by the development team to track incomplete tasks, and assign them to particular team members. It is connected to our existing GitHub accounts, which provides ease of access, making team members more likely to check in with the task list.

Google Drive

Google Drive was adopted for its strengths in cloud-based storage and integration with Google Docs, supporting the team's need for synced and accessible documentation. Its real-time collaboration allows for parallel contribution to project documentation, aligning with Agile's collaborative principles. Version history provides the team with a safeguard against accidental overwrites, and improves accountability within tracking document changes. We preferred Google Drive over alternatives like Microsoft OneDrive for its more intuitive web interface, smoother editing experience, and integration with the university's Google Workspace. Using google docs, we were able to add comments and suggestions to better coordinate our workflow.

2D Design

Two-dimensional graphics were chosen for their mutual benefits to both developers and players. They are suitable for developers new to gamemaking as when compared to three dimensions, a reduced field to plan, design, and code for makes development easier. For video-game beginners, two dimensions are conceptually easier to interact with, as they require fewer physical input mappings to negotiate when moving around an environment. A reduced dimension is also less resource intensive to build and run, improving developer experience with shorter build times and more responsive playtests; further aligning our game with the client's expectations for it to be cross-platform and run without specialist hardware, as a 2D game should run across all platforms without performance issues.

Game Engine

We chose libGDX as our game engine due to its rich documentation, community support, and extensibility when producing 2D games. The community maintains

tutorials and accessible forums for beginners to ask questions and receive feedback, supporting our implementation team who have never used the engine before. It is cross-platform and has low overhead, allowing us to satisfy our requirement for the game to work on a variety of devices, including low-powered systems. An alternative approach would have been to use jMonkeyEngine or to interface with the Lightweight Java Game Library directly, instead of through the structured layer which is libGDX. However, these choices would have been illogical for us as the complex development challenges these alternatives bring would have been unmanageable and unnecessary within the scope of this project.

JUnit

We chose JUnit to implement our automated tests because it is a standard tool in the Java ecosystem. This improves the maintainability of our code base, since future developers are more likely to have experience with JUnit. There is also a good range of documentation available both for JUnit generally, and its usage with libGDX, which made it a convenient choice.

JaCoCo

We chose JaCoCo to generate code coverage reports because it has good integration with GitHub Actions - this means that reports can be automatically generated and made accessible to all team members. This is beneficial because it means that coverage data can be accessed without needing a development environment installed. It provides an easy way for all team members to see how much testing has been done, which is useful for project management.

Text Editor / IDE

Team members were permitted to use whichever editor they preferred, because all of our code is standard Java, not tied to a particular environment. This was beneficial because it meant that developers could use an environment they are already familiar with. We also had a mix of Windows and Linux machines, so standardising on a particular software suite could pose a difficulty if it was only available on one platform. The Text Editor / IDEs we used were Vim, VS code and IntelliJ.

Whatsapp

We used whatsapp as a primary communication method as it allowed us to keep in touch and confirm dates for meetings and expectations for work outside of the meetings. If one of us had a problem with anything in the workflow we could easily communicate with one another as a risk mitigation shown in R2 and R6. We also used whatsapp to discuss our project.

Team Organisation methods:

The first part of our working structure that we implemented were weekly team meetings with the whole group to cover updates to the project and allow for communication within the whole team. This is because it allows the whole team to get a view of what is happening within the project and for us to be able to update plans with every team member's input. It also suited the group very well as it allowed for reassurance and clarity on every part of the project getting completed.

The next large working structure that we implemented was splitting into smaller groups and splitting the deliverables between those groups. This meant that every work package and deliverable had its own group with at least one person and one backup team member for the smallest work packages, for example the website, and all team members on the larger or more collaborative work packages, for example testing. This is because it allows the deliverables to be completed with less overheads for assigning individual tasks to people and it reduces any potential confusion with what people are working on. In addition, we allowed groups to work autonomously, with exception for the three weekly meetings on Mondays, Wednesdays and Fridays with the whole group, so they could meet if necessary or just work online depending on the task and it wouldn't effect or be reliant on the whole teams availability allowing for individual tasks to be completed in a more agile and effective way.

Evolution of plan

Our plan initiated with dividing the team into two sub-teams - the technical team and the non technical team. We created a clear list of appropriate tasks needed to fulfil all assessment criteria, which made it easier for task delegation with a set of tasks delegated to each team member, as per each individual's strengths and preferences. Over time we found that different tasks required varying levels of time. An example being T1.4 as most risks were the same. Comparing this to T4, which required T3.1 to be completed. This made the previous task system inefficient. So we decided to move onto a new task system which labeled each task out with an ID along with any dependencies needed to complete the task. You can see this task list below.

Tasks

Task ID	Name	Description	Start	End	Dependancy
T1.1	Update deliverable - Requirements	Update the new requirements for the second part of the project	27/11/2025	4/12/2025	N/A
T1.2	Update deliverable - Architecture	Upload and and update any and all changes to the architecture	27/11/2025	4/12/2025	N/A
T1.3	Update deliverable - Method Selection and Planning	Update the new methods and tools used for the second part of the project and tasks	27/11/2025	4/12/2025	N/A

T1.4	Update deliverable - Risk assessment	Assess and document any new risks from the new assessment	27/11/2025	4/12/2025	N/A
T2.1	Change report pt A	Summarise the processes, tools and conventions your team used to plan, make, keep track of, and review changes to the Assessment 1 deliverables	04/12/2025	05/12/2025	T1
T2.2.1	Change report pt B - Requirements	Describe and list out any of the changes to the requirements giving reasons. Give Requirement codes	04/12/2025	06/12/2025	T1.1
T2.2.2	Change report pt B - Architecture	Describe and list out any of the changes to the architecture giving reasons for the changes.	11/12/2025	09/01/2026	T1.2, T3.1
T2.2.3	Change report pt B - Method selection and planning	Describe and list out any of the changes to the Method selection and planning giving the new tools and describing the task structure compared to the old version.	11/12/2025	11/12/2025	T1.3

T2.2.4	Change report pt B - Risk assessment	Describe and list out any of the changes to the Risk assessment giving reasons behind any new risk made. Write out the changed risk numbers	11/12/2025	11/12/2025	T1.4
T3.1	Implementation - updated code	Make sure to highlight how code has changed compared to the previous group, consistent with change report	4/12/2025	11/12/2025	T1.1
T3.2.1	Implementation - Show any changes to software licensing used	Check any new software used and update the licensing.	11/12/2025	09/01/2026	T3.1
T3.2.2	Implementation - List out any requirements not met	Write in the events implemented and go through requirements for anything that is not implemented.	11/12/2025	09/01/2026	T3.1
T4.1	Software Testing Report - Summary	Briefly summarise your testing method(s) and approach(es), explaining why these are appropriate for the project.	11/12/2025	09/01/2026	T3.1
T4.2	Software Testing Report -	Give a brief report on the	11/12/2025	09/01/2026	T3.1

	brief report on the tests	actual tests, including statistics of what tests were run and what results were achieved, with a clear statement of any tests that are failed by the current implementation			
T4.3	Software Testing Report - Links to data used and description	Provide the precise URLs for the testing material on the website: this material should comprise the testing results and coverage report generated by your automated testing tooling, and descriptions of manual test-cases	11/12/2025	09/01/2026	T3.1
T5.1	User Evaluation Report - Brief method explanation	A brief report of the method for the user evaluation (e.g. recruitment, data collection tools and data procedures), explaining why these were chosen.	11/12/2025	07/01/2026	T3.1
T5.2	User Evaluation Report - Table	A table listing the usability	11/12/2025	07/01/2026	T5.1

	report with data, and ethics forms	problems found by users in the prototype system and the users' severity ratings of those problems. Ensure ethics forms are done			
T6.1	Continuous Integration Report - Summary of your continuous integration (CI) method(s) and approach(es)	Summarise your continuous integration (CI) method(s) and approach(es), explaining why these are appropriate for the project	11/12/2025	07/01/2026	T2, T3.1
T6.2	Continuous Integration Report - Brief report on continuous integration infrastructure	Give a brief report on the actual continuous integration infrastructure you have set up for your project. You may use additional pages for a bibliography. Any other content that overruns the page limit will not be considered by the markers and will not receive any marks.	11/12/2025	07/01/2026	T6.1